Documentation Project 2: BrewBuddy Pro

Group members: Jeroen Breedveld, Daan van Veen, Omar Ba Slaim

Running Instructions:

Please follow these steps to successfully run the program:

1. Unpack the .zip file: Ensure all files (including the Python script and any .txt or .csv files) are located in the exact same directory.
2. Fill in the Google Form: Have participants sign up using this link:

https://docs.google.com/forms/d/e/1FAIpQLSd6CeVgLXvRE5YGlkfYEkanezgUMk_-0mmGZOGv39igk2e7Lg/viewform?usp=dialog

3. Navigate to the linked Google Sheets and download the data as a .csv file:

https://docs.google.com/spreadsheets/d/1PpeRo5NwWtCWgkyHn9dRoUrztqWxIX0mFlPERe4WtIU/edit?usp=sharing

4. Move the downloaded .csv file into the same directory as the Python script. *(Tip: Remember to clear the Google Sheet responses after running the script to ensure a clean slate for the next round!)*
5. Execute the Python script. The script will output the generated groups in the terminal, save the pairings to a .csv file for history tracking, and automatically generate a .txt file containing the personalized messages and conversation starters for all matched participants.

Troubleshooting / Error Handling:

If the script cannot automatically locate the .csv file, a built-in guide will appear in the terminal. The program will print the following instructions to guide the user:

CSV file not found.

Follow these steps:

1. Fill in the Google Form

2. Download the responses as CSV

3. Place the CSV file in the same folder as this script

Press Enter when ready...

Note: The script is designed to automatically recognize common default filenames from Google Sheets. As a backup, it will also prompt the user to manually enter the filename in the terminal if the automated search fails.

Project Status

All basic project requirements are completed (100%). This includes dynamic CSV reading, configurable group sizes/rounds, residual group handling, random matching, conversation starters, and output formatting.

Beyond the basic requirements, we also implemented the following features to enhance the program:

- We created an option to generate multiple rounds of matches. This way it is easier to plan ahead if you're reusing the same list of people.
- We integrated a history-tracking system (saving and reading from an all_pairs.csv file). Our code checks past iterations to ensure that the exact same groups are not matched together again in subsequent rounds, which makes the lottery much more effective over time.
- Our code features robust error handling. It automatically searches for common Google Forms CSV filenames. If it fails, it provides an menu to guide the user, alongside input validation for group sizes and round numbers to prevent the program from breaking.

Known Issues & Limitations:

- While the program successfully generates and saves new matches for the current round (new_pairs.csv), we encountered a bug in our final version regarding the long-term history tracking. Appending the newly matched pairs to the all_pairs.csv file does not always work as intended. We have left the code intact for transparency and future debugging, but as a result, the prevention of redundant matchings across *multiple sessions* may not function perfectly if the user generates multiple rounds per run.

Group Members & Task Division

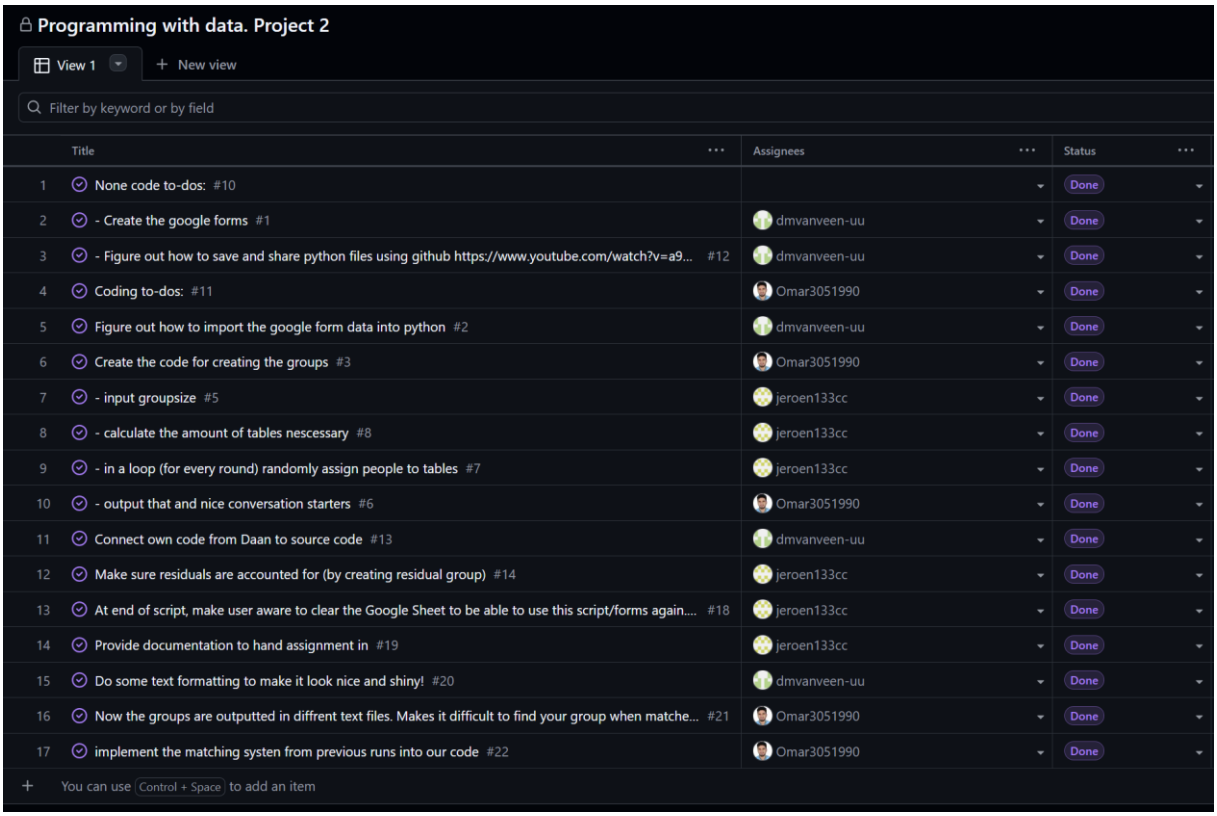
Members: Daan, Jeroen, and Omar.

Note: All listed members participated, no members dropped out. We are not INFOMCTH students.

Task Division:

We kept track of our specific tasks using a GitHub project board (see the attached screenshot of our To-Do list) and a Changelog. You can find the exact breakdown of who wrote which part of the code in the Changelog at the top of our Python script. Broadly speaking:

- **Daan:** CSV imports, merging code, and input error handling.
- **Jeroen:** In-program instructions, residual group logic, final debugging and documentation.
- **Omar:** Core random matching algorithm and data logic.



Programming with data. Project 2

View 1 | + New view

Filter by keyword or by field

	Title	Assignees	Status
1	None code to-dos: #10		Done
2	- Create the google forms #1	dmvanveen-uu	Done
3	- Figure out how to save and share python files using github https://www.youtube.com/watch?v=a9... #12	dmvanveen-uu	Done
4	Coding to-dos: #11	Omar3051990	Done
5	Figure out how to import the google form data into python #2	dmvanveen-uu	Done
6	Create the code for creating the groups #3	Omar3051990	Done
7	- input groupsize #5	jeroen133cc	Done
8	- calculate the amount of tables nescessary #8	jeroen133cc	Done
9	- in a loop (for every round) randomly assign people to tables #7	jeroen133cc	Done
10	- output that and nice conversation starters #6	Omar3051990	Done
11	Connect own code from Daan to source code #13	dmvanveen-uu	Done
12	Make sure residuals are accounted for (by creating residual group) #14	jeroen133cc	Done
13	At end of script, make user aware to clear the Google Sheet to be able to use this script/forms again.... #18	jeroen133cc	Done
14	Provide documentation to hand assignment in #19	jeroen133cc	Done
15	Do some text formatting to make it look nice and shiny! #20	dmvanveen-uu	Done
16	Now the groups are outputted in diffrent text files. Makes it difficult to find your group when matche... #21	Omar3051990	Done
17	implement the matching systen from previous runs into our code #22	Omar3051990	Done

+ You can use Control + Space to add an item

Git / GitHub & Workflow

Because we are not INFOMCTH students, using Git/GitHub was not strictly mandatory for us, but we made a serious effort to implement it.

- **Public Repository:** https://github.com/dmvanveen-uu/Daan-repository/tree/main
- **Project Board:** https://github.com/users/dmvanveen-uu/projects/2

(Note: The repository is currently public for grading purposes but will be made private again afterward.)

Process & Approach

Our approach to tackling this assignment evolved significantly as we progressed. Initially, Daan started coding without using the provided base file for reading CSVs. Because the relevant lecture on file I/O had not yet taken place, this required a considerable amount of research and Googling.

During a subsequent seminar, we sat down to review our initial code and determine the best path forward. We found the original provided script to be rather complicated to adapt. To make our workflow more efficient, we made the strategic decision to start from scratch, selectively copying only the strictly necessary parts from the original code. This allowed us to build a clean foundation without getting tangled in legacy code we didn't fully understand.

With a clean slate, we mapped out our "need-to-haves" versus "nice-to-haves" (such as the API integrations) and divided the tasks. We worked on our sections separately, but continuously built upon each other's code commits. A week later, we held a digital video call to discuss the final integrations, identify any necessary fixes, and set priorities for our remaining time. A major focus of this meeting was brainstorming how to cleanly implement the "multiple rounds" function and ensuring our custom script successfully accounted for duplicate matchings (a feature present in the original code that we needed to properly replicate in our new version).

Workflow Reflection:

We worked with separate versions that we uploaded manually via commits. To maintain a clear overview, we kept a changelog in the Python script and utilized the GitHub Projects board to track our tasks. This ensured it was always clear who did what and who to ask if we didn't understand a piece of code. We found this overview very helpful. For future projects, we absolutely want to carry this forward, but integrate it using the official Git workflow (working with proper branches and merges directly).

Use of Generative AI

Our use of Generative AI in this project was minimal and strictly facilitative. We used GenAI for three things:

1. GitHub Crash Course: We asked AI for a quick crash course on Git/GitHub. While the explanation was good, we concluded that (given the time constraints and our reliance on WhatsApp for remote communication) it would cause too much friction to fully implement standard branching for this specific project.

2. Feature brainstorm: We asked AI to think of some features which would help this project to a higher level. We discussed the output in a online meeting. We dediced to do the ‘multiple rounds’ implementation and check whether the API connections for mailing and pulling the Google Forms is within our capabilities.
3. API Feasibility Check: We briefly used AI to research integrating a Google Forms API as an extra feature. We concluded that we did not feel comfortable enough with this yet. Getting it to work would have required having AI write significant portions of the code. Because this goes directly against the rules of the assignment, we decided to drop this feature entirely.